Программирование в командном процессоре ОС UNIX. Командные файлы

Лабораторная работа № 12 (зачётная книжка № 1132259051), РУДН

Чжан Вэньцзюнь

2026-10-20

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**
4.1 1. Скрипт резервного копирования самого себя 9
4.2 2. Скрипт обработки произвольного числа аргументов 10
4.3 3. Аналог команды ls 12
4.4 4. Подсчёт файлов заданного формата 12

5 Выводы **15**

6 Ответы на контрольные вопросы **16**
6.1 1. Понятие командной оболочки, примеры и различия 16
6.2 2. Что такое POSIX? 16
6.3 3. Как определяются переменные и массивы в bash? 17
6.4 4. Назначение операторов let и read 17
6.5 5. Арифметические операции в bash 17
6.6 6. Что означает операция (( ))? 17
6.7 7. Стандартные имена переменных 18
6.8 8. Что такое метасимволы? 18
6.9 9. Как экранировать метасимволы? 18
6.10 10. Как создавать и запускать командные файлы? 18
6.11 11. Как определяются функции в bash? 19
6.12 12. Как выяснить, является файл каталогом или обычным файлом? . 19
6.13 13. Назначение команд set, typeset и unset 19
6.14 14. Как передаются параметры в командные файлы? 20
6.15 15. Специальные переменные bash и их назначение 20

Список литературы **21**

Список иллюстраций

4.1 Работа скриптов backup.sh и args.sh 11
4.2 Работа скриптов myls.sh и count.sh 14

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

2 Задание

В соответствии с п. 10.3 методических указаний необходимо написать четыре командных файла:

1. Скрипт, который при запуске делает резервную копию **самого себя** (файла с собственным исходным кодом) в каталог `backup` в домашнем каталоге; файл должен архивироваться одним из архиваторов (`zip`, `bzip2` или `tar`) — способ использования команд архивации изучить по справке.
2. Пример командного файла, обрабатывающего произвольное число аргументов командной строки, в том числе превышающее десять (последовательно печатает значения всех переданных аргументов).
3. Командный файл — аналог команды `ls` (без использования самой команды `ls` и команды `dir`), выдающий информацию о нужном каталоге и о правах доступа к файлам этого каталога.
4. Командный файл, получающий в качестве аргумента командной строки формат файла (`.txt`, `.doc`, `.jpg`, `.pdf` и т. д.) и вычисляющий количество таких файлов в указанной директории; путь к директории также передаётся аргументом.

3 Теоретическое введение

Командный процессор (оболочка, интерпретатор команд, shell) — программа, позволяющая пользователю взаимодействовать с операционной системой. Основные реализации: оболочка Борна `sh`, C-оболочка `csh`, оболочка Корна `ksh` и `bash` (Bourne Again Shell), совмещающая свойства `csh` и `ksh`. Стандарт **POSIX** определяет интерфейсы взаимодействия ОС и прикладных программ для переносимости на уровне исходного кода [1].

В `bash` переменные имеют строковый тип: `mark=/usr/andy/bin`; подстановка значения — `${mark}`. Массивы создаются командой `set -A имя значения…`, индексация с нуля. Арифметика выполняется командой `let` (`let sum=x+7`) или конструкцией `(( ))`; доступны операции `+`, `-`, `*`, `/`, `%`, сравнения, побитовые, присваивания с операцией (`+=`, `-=` и т. д.). Ввод в переменную — команда `read`.

Метасимволы (`*`, `?`, `[...]`, `~`, `$`, `;`, `&`, `|`, `>`, `<`, `(`, `)`, `\`, `"`, `'`) имеют специальный смысл; экранируются обратной косой чертой (`\*`), одинарными кавычками (полное экранирование) или двойными кавычками (экранирование всех, кроме `$`, `` ` `` и `\`).

Командный файл — текстовый файл с последовательностью команд, запускаемый как программа; ему выдаётся право на исполнение (`chmod +x`). Параметры доступны через `$1`, `$2`, …, `$#` (число), `$*`/`$@` (все), `$0` (имя файла); сдвиг — `shift`. Специальные переменные: `$?` (код возврата), `$$` (PID), `$!` (PID последнего фонового), `$IFS`, `$HOME`, `$PATH`, `$PS1`, `$PS2`, `$RANDOM`.

Управляющие конструкции: `if … then … elif … else … fi`, `for имя in список; do … done`, `while …; do … done`, `until …; do … done`, `case … in … esac`, прерывание циклов — `break` и `continue`. Функции: `имя() { команды; }`.

Параметры обрабатываются командой `getopts`. Проверка типа файла — `test -f`, `test -d` (или `[ -f ]`, `[ -d ]`).

4 Выполнение лабораторной работы

Работа выполнена в виртуальной машине VirtualBox (Fedora Linux 44, `bash 5.3.9`), все командные файлы созданы в каталоге `~/work/os/lab10` и сделаны исполняемыми (`chmod +x`).

4.1 1. Скрипт резервного копирования самого себя

Листинг `backup.sh`:

```
#!/bin/bash
# резервная копия собственного исходного кода в ~/backup (tar)
DEST="$HOME/backup"
mkdir -p "$DEST"
NAME="$(basename "$0")"
STAMP=$(date +%Y%m%d-%H%M%S)
tar -czf "$DEST/${NAME}.${STAMP}.tar.gz" "$0"
echo "Архив создан: $DEST/${NAME}.${STAMP}.tar.gz"
ls -lh "$DEST" | tail -3
```

Способ архивации изучен по `man tar`: опция `-c` создаёт архив, `-z` включает сжатие gzip, `-f` задаёт имя файла. Ключевой приём — параметр `$0` содержит имя запущенного файла, поэтому `tar -czf … "$0"` архивирует сам исходный код скрипта.

Результат запуска:

```
Архив создан: /home/vzhang/backup/backup.sh.20261020-135602.tar.gz
итого 8,0K
-rw-r--r--. 1 vzhang vzhang 376 окт 20 13:55 backup.sh.20261020-
135549.tar.gz
-rw-r--r--. 1 vzhang vzhang 376 окт 20 13:56 backup.sh.20261020-
135602.tar.gz
```

Содержимое архива проверено командой `tar -tzf ~/backup/*.tar.gz` — внутри находится `./backup.sh`.

4.2 2. Скрипт обработки произвольного числа аргументов

Листинг `args.sh`:

```
#!/bin/bash
echo "Всего аргументов: $#"
i=1
for a in "$@"; do
  echo "  аргумент $i = $a"
  i=$((i+1))
done
shift 2
echo "После shift 2 осталось: $#"
for a in "$@"; do echo "  остаток: $a"; done
```

Запуск с двенадцатью аргументами:

```
Всего аргументов: 12
  аргумент 1 = a1
```

```
  …
  аргумент 12 = a12
После shift 2 осталось: 10
  остаток: a3
  …
  остаток: a12
```

Цикл `for a in "$@"` перебирает все аргументы независимо от их числа (в том числе больше десяти), `$#` даёт их количество, `shift 2` сдвигает список на два элемента. Результаты пп. 1–2 показаны на рис. 4.1.

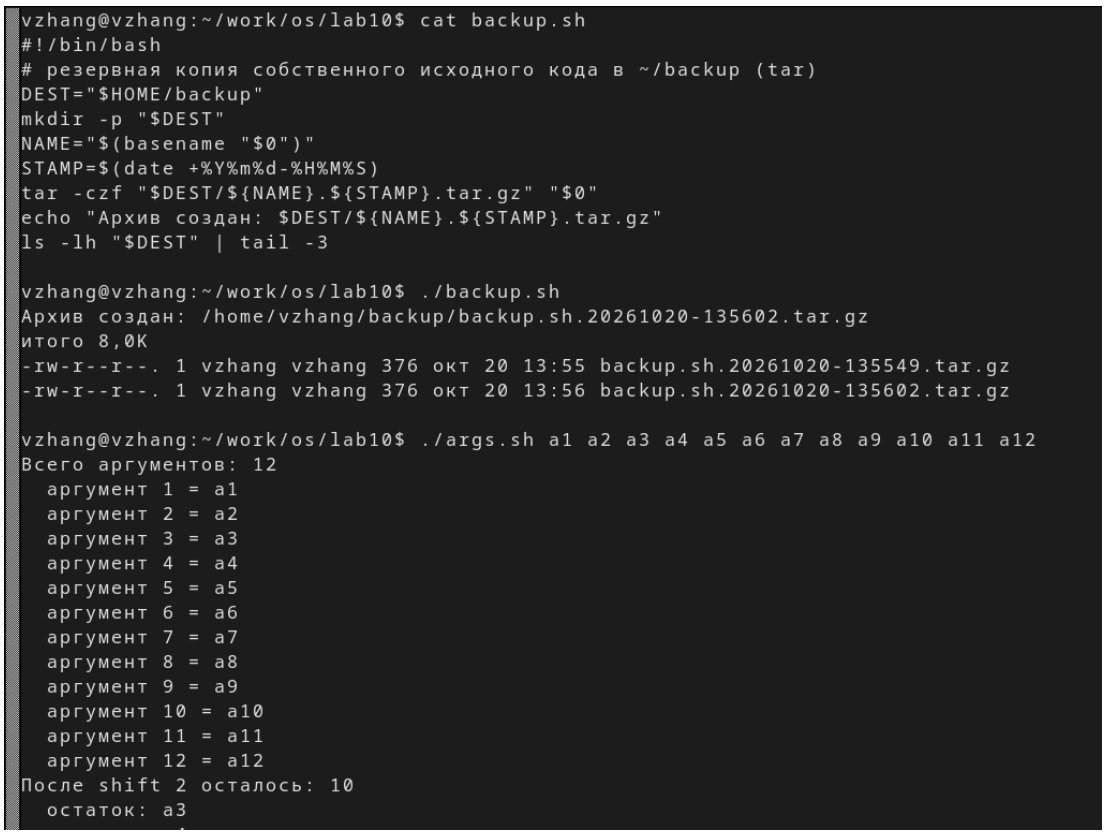


Рисунок 4.1: Работа скриптов backup.sh и args.sh

4.3 3. Аналог команды ls

Листинг `myls.sh` (команды `ls` и `dir` не используются — применяются подстановка по маске и команда `stat`):

```
#!/bin/bash
# аналог ls без использования команд ls и dir
DIR="${1:-.}"
[ -d "$DIR" ] || { echo "Не каталог: $DIR" >&2; exit 1; }
echo "Каталог: $(cd "$DIR" && pwd)"
for f in "$DIR"/* "$DIR"/.[!.]*; do
  [ -e "$f" ] || continue
  printf '%-48s %s\n' "$(basename "$f")" "$(stat -c '%A %U:%G %6s %y' "$f")"
done
```

Результат:

```
Каталог: /home/vzhang/work/os/lab10
args.sh          -rwxr-xr-x vzhang:vzhang    238 2026-10-20 13:55:49
backup.sh        -rwxr-xr-x vzhang:vzhang    338 2026-10-20 13:55:49
count.sh         -rwxr-xr-x vzhang:vzhang    299 2026-10-20 13:55:49
myls.sh          -rwxr-xr-x vzhang:vzhang    355 2026-10-20 13:55:49
```

Формат `%A` команды `stat` выводит права доступа в символьной форме (`-rwxr-xr-x`), `%U:%G` — владельца и группу, `%s` — размер, `%y` — время изменения. Скрипт выводит и скрытые файлы (маска `.[!.]*`).

4.4 4. Подсчёт файлов заданного формата

Листинг `count.sh`:

```
#!/bin/bash
EXT="${1:?укажите формат (txt, sh, png ...)}"
DIR="${2:-.}"
[ -d "$DIR" ] || { echo "Не каталог: $DIR" >&2; exit 1; }
n=0
for f in "$DIR"/*."$EXT"; do
  [ -f "$f" ] && n=$((n+1))
done
echo "Файлов с расширением .$EXT в каталоге $DIR: $n"
```

Результаты:

```
Файлов с расширением .sh в каталоге .: 4
Файлов с расширением .png в каталоге /home/vzhang/projects/lab10/image: 3
```

Конструкция `${1:?сообщение}` завершает скрипт с сообщением, если первый аргумент не задан; `${2:-.}` использует текущий каталог по умолчанию. Результаты пп. 3–4 показаны на рис. 4.2.

```
vzhang@vzhang:~/work/os/lab10$ cat myls.sh
#!/bin/bash
# аналог ls без использования команд ls и dir
DIR="${1:-.}"
[ -d "$DIR" ] || { echo "Не каталог: $DIR" >&2; exit 1; }
echo "Каталог: $(cd "$DIR" && pwd)"
for f in "$DIR"/* "$DIR"/.[!.]*; do
  [ -e "$f" ] || continue
  printf '%-48s %s\n' "$(basename "$f")" "$(stat -c '%A %U:%G %6s %y' "$f")"
done

vzhang@vzhang:~/work/os/lab10$ ./myls.sh ~/work/os/lab10
Каталог: /home/vzhang/work/os/lab10
args.sh                                          -rwxr-xr-x vzhang:vzhang    238 202
backup.sh                                        -rwxr-xr-x vzhang:vzhang    338 202
count.sh                                         -rwxr-xr-x vzhang:vzhang    299 202
myls.sh                                          -rwxr-xr-x vzhang:vzhang    355 202

vzhang@vzhang:~/work/os/lab10$ cat count.sh
#!/bin/bash
EXT="${1:?укажите формат (txt, sh, png ...)}"
DIR="${2:-.}"
[ -d "$DIR" ] || { echo "Не каталог: $DIR" >&2; exit 1; }
n=0
for f in "$DIR"/*."$EXT"; do
  [ -f "$f" ] && n=$((n+1))
done
echo "Файлов с расширением .$EXT в каталоге $DIR: $n"

vzhang@vzhang:~/work/os/lab10$ ./count.sh sh .
Файлов с расширением .sh в каталоге .: 4

vzhang@vzhang:~/work/os/lab10$ ./count.sh png ~/projects/lab10/image
```

Рисунок 4.2: Работа скриптов myls.sh и count.sh

5 Выводы

В ходе работы изучены основы программирования в оболочке `bash`: переменные и их подстановка, арифметика (`$(( ))`, `let`), метасимволы и их экранирование, передача параметров (`$1`, `$#`, `$@`, `shift`) и специальные переменные (`$0`, `$?`, `$RANDOM`), управляющие конструкции (`for`, `if`, `while`, `case`). Написаны и проверены четыре командных файла: скрипт резервного копирования самого себя в `~/backup` архиватором `tar` с использованием `$0`; скрипт обработки произвольного (в том числе более десяти) числа аргументов; аналог команды `ls`, не использующий `ls`/`dir` и выводящий права доступа через `stat`; скрипт подсчёта файлов заданного формата в указанном каталоге. Поставленные задачи выполнены полностью.

6 Ответы на контрольные вопросы

6.1 1. Понятие командной оболочки, примеры и различия

Командная оболочка — программа-интерпретатор, обеспечивающая взаимодействие пользователя с ОС: читает команды, выполняет их (встроенные или внешние программы), поддерживает переменные, перенаправления, конвейеры, управляющие конструкции и сценарии. Примеры: `sh` (Борна — базовый полный набор функций), `csh` (C-подобный синтаксис, история, но неудобен для скриптов), `ksh` (совместим с Борном, добавлены арифметика и расширения), `bash` (совмещает `csh` и `ksh`, GNU), `zsh` (расширенный, автодополнение), `fish`. Различия — в синтаксисе, наборе встроенных команд, возможностях сценариев, обработке истории и автодополнении.

6.2 2. Что такое POSIX?

POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов IEEE, описывающих интерфейсы взаимодействия ОС и прикладных программ (системные вызовы, утилиты, оболочку, потоки, файловые системы) для совместимости UNIX/Linux-подобных систем и переносимости программ на уровне исходного кода. POSIX-совместимые оболочки построены на базе оболочки Корна.

6.3 3. Как определяются переменные и массивы в bash?

Переменная присваивается без пробелов вокруг `=`: `name=value`; значение — строка. Использование — `$name` или `${name}`. Экспорт в среду потомков — `export name`. Массив: `arr=(one two three)` или `set -A arr one two three`; элемент — `${arr[0]}`, все — `${arr[@]}`, длина — `${#arr[@]}`; индексация с нуля; присваивание элементу — `arr[3]=value`.

6.4 4. Назначение операторов let и read

`let` — вычисление арифметического выражения и присваивание результата переменной (`let x=x+7`, `let "y = a * b"`); имя переменной в выражении пишется без `$`. `read` — чтение строки из стандартного ввода в переменные: `read name`, `read -p "Введите: " var`, `read -a arr`, `read -r` (без обработки `\`).

6.5 5. Арифметические операции в bash

`+`, `-`, `*`, `/`, `%` (остаток), унарные `-`, `+`, `!`; сравнения `==`, `!=`, `<`, `<=`, `>`, `>=`; побитовые `&`, `|`, `^`, `~`, `<<`, `>>`; логические `&&`, `||`; присваивания с операцией `+=`, `-=`, `*=`, `/=`, `%=` и др.; инкремент/декремент `++`, `--`; возведение в степень `**`; запись в других системах счисления — `radix#number` (2, 8, 16).

6.6 6. Что означает операция (( ))?

`(( выражение ))` — арифметическое вычисление в стиле C: возвращает код возврата 0 («истина»), если значение выражения не равно нулю, и 1 — если равно нулю. Используется в условиях: `if (( x > 0 )); then …`, для присваивания: `(( x`

`= a + b ))`, инкремента: `(( i++ ))`. Внутри `(( ))` имена переменных пишутся без `$`.

6.7 7. Стандартные имена переменных

`HOME`, `PATH`, `PWD`, `OLDPWD`, `USER`, `LOGNAME`, `UID`, `GROUP`, `SHELL`, `TERM`, `LANG/LC_*`, `PS1`, `PS2`, `IFS`, `RANDOM`, `LINENO`, `SECONDS`, `HOSTNAME`, `HOSTTYPE`, `OSTYPE`, `PPID`, `?`, `$`, `!`, `#`, `@`, `*`, `0–9`.

6.8 8. Что такое метасимволы?

Метасимволы — символы, имеющие специальный смысл для оболочки и не передаваемые программе буквально: `*`, `?`, `[...]` (шаблоны имён), `~` (домашний каталог), `$` (подстановка), `` ` `` и `$( )` (подстановка команды), `;`, `&`, `|`, `>`, `<`, `>>`, `2>`, `(`, `)`, `{`, `}`, `\`, `"`, `'`, `#` (комментарий).

6.9 9. Как экранировать метасимволы?

Обратной косой чертой: `\$`, `\*`, `\`, `\;` — символ становится литералом. Одинарными кавычками `'…'` — экранируется всё, включая `$`. Двойными кавычками `"…"` — экранируются все метасимволы, кроме `$`, `` ` `` и `\`. Также применяется «здесь-документ» с кавычками (`<<'EOF'`) для запрета подстановок.

6.10 10. Как создавать и запускать командные файлы?

Создать текстовый файл (любым редактором) с первой строкой-шебангом `#!/bin/bash`, записать команды, выдать право на исполнение `chmod +x file` и

запустить `./file` (или `bash file`). Если каталог файла входит в PATH, достаточно указать имя. Аргументы передаются после имени: `./file arg1 arg2`.

6.11 11. Как определяются функции в bash?

```
имя() {
    команды
    return код     # необязательно
}
# или
function имя { команды; }
```

Вызов — по имени с аргументами (`имя a b`); внутри доступны `$1`, `$#`, `$@`; результат — через `echo` или `return` (код 0–255), получить его можно через `$?`.

6.12 12. Как выяснить, является файл каталогом или обычным файлом?

Командой `test` (или `[ ]`): `[ -d путь ]` — истина для каталога; `[ -f путь ]` — для обычного файла; `[ -L ]` — символическая ссылка; `[ -e ]` — существует. Также `stat` (поле типа), `file`. В скрипте: `if [ -d "$1" ]; then echo каталог; elif [ -f "$1" ]; then echo файл; fi`.

6.13 13. Назначение команд set, typeset и unset

- `set` — вывод и изменение параметров оболочки: `set -x` (трассировка), `set -e` (выход при ошибке), `set -u` (ошибка при неопределённой переменной), `set -- аргументы` (установка позиционных параметров), `set -A` (массив, в ksh/bash).

- `typeset` (в bash — синоним `declare`) — объявление переменных с атрибутами: `typeset -i x` (целая), `-r` (только чтение), `-a` (массив), `-x` (экспорт), `-f` (список функций).
- `unset` — удаление переменной или функции: `unset name`, `unset -f func`.

6.14 14. Как передаются параметры в командные файлы?

После имени файла через пробел: `./script a b c`. Внутри доступны `$1`, `$2`, …; с 10-го — `${10}`, `${11}` (иначе `$10` читается как `$1` + `0`). `$#` — количество, `$*`/`$@` — все параметры (различие: `"$@"` — каждый в отдельной кавычке, `"$*"` — одной строкой), `shift n` — сдвиг. Разбор ключей — `getopts`.

6.15 15. Специальные переменные bash и их назначение

- `$0` — имя запущенного файла/оболочки;
- `$1`…`$9`, `${10}`… — позиционные параметры;
- `$#` — число параметров;
- `$*`, `$@` — все параметры;
- `$?` — код возврата последней команды;
- `$$` — PID текущей оболочки;
- `$!` — PID последнего фонового процесса;
- `$-` — текущие флаги оболочки;
- `$_` — последний аргумент предыдущей команды;
- `$IFS` — разделители полей;
- `$RANDOM`, `$LINENO`, `$SECONDS`, `$PPID`, `$UID`, `$EUID`.

Список литературы

1. Robbins A. Bash Pocket Reference. O'Reilly Media, 2016. 156 c.